

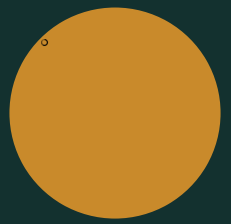
Variables & Data Types

The Building Blocks of Every Python Program

Session 3

What We're Covering Today

- What is Conda, and what is a virtual environment?
- Basic Conda commands — create, list, activate, delete
- Installing packages with conda vs. pip
- Your first program — the print() statement
- What is a variable? (and how its value can change)
- Rules for naming a variable
- Different naming styles — snake_case, camelCase & more
- The four core data types
- Checking a type with type()
- Type casting — and why input() needs it

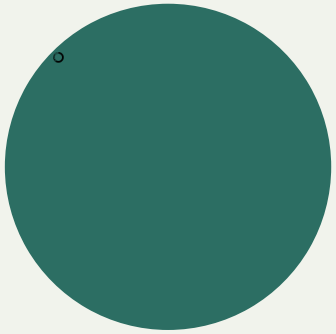


PART 1

Conda & Virtual Environments

Before we write any Python — let's set the stage properly.

What Is Conda?



Manages Python and your tools

- Conda is a tool that installs Python — and other tools — on your computer
- It also creates separate, clean workspaces for each project, called environments
- Miniconda is simply the small, lightweight way to install Conda
- Think of Conda as the manager in charge of every toolbox you own

What Is a Virtual Environment?



A separate, clean workspace

- A virtual environment is its own private workspace, built for one project
- It has its own Python version and its own installed packages, separate from every other project
- This stops one project's tools from clashing with another project's tools
- Analogy: separate toolboxes for separate jobs, so tools never get mixed up

Creating & Activating an Environment

- conda create makes a brand-new environment
- You choose the name, and which Python version it should use
- conda activate switches you "into" that environment
- You only need to create an environment once per project — but you activate it every time you work

```
# create a new environment named "myenv" with Python 3.11
conda create --name myenv python=3.11

# activate it
conda activate myenv

# your terminal prompt now shows (myenv)
```

Viewing & Removing Environments

- conda env list shows every environment on your computer
- The one with a star (*) next to it is currently active
- conda deactivate exits the current environment
- conda env remove permanently deletes one — use this carefully

```
# see every environment you have
conda env list

# leave the current environment
conda deactivate

# delete an environment completely
conda env remove --name myenv
```

Conda Command Cheat Sheet

Create

conda create --name myenv python=3.11

Activate

conda activate myenv

List

conda env list

Delete

conda env remove --name myenv

Installing Packages With Conda

- Packages are extra tools other people built, ready for you to use
- Once inside an environment, conda install adds a package to it
- You can install several packages in one line

```
# make sure your environment is active first
conda activate myenv

# install one package
conda install numpy

# install several at once
conda install pandas matplotlib
```

Installing Packages With pip

- pip is Python's own built-in package installer
- It installs from PyPI — the massive public library of Python packages
- pip works perfectly fine inside a conda environment too

```
# make sure your environment is active first
conda activate myenv

# install one package
pip install numpy

# install several at once
pip install pandas matplotlib
```

Conda vs. pip — What's the Difference?

Conda

- Manages both environments AND packages
- Can install non-Python tools too (e.g. R, C libraries)
- The standard choice for data science stacks
- Slightly slower to install packages

pip

- Manages Python packages only — not environments
- Installs from PyPI, the largest Python package library
- Often has the very newest package versions first
- Works inside a conda environment without any conflict



PART 2

Let's Write Some Code

Every programmer starts in the exact same place.

Hello, World!



Every coder's first program

- print() is a function that displays something on the screen
- Almost every programmer, in almost every language, writes this exact line first
- Type this in VS Code, in the environment you set up last class:
- print("Hello, World!")

What Can print() Accept?

- Text (in quotes), whole numbers, and decimal numbers all work
- Separate several things with commas — print adds a space between each one automatically
- An empty print() just prints a blank line — useful for spacing things out

```
print("Hello, World!")
print(5)
print(3.14)

print("My favorite number is", 7)

print()  # prints a blank line
```



PART 3

What Is a Variable?

A Variable Is a Labeled Container



Think of a bottle

- Imagine a bottle with a label on it, like "age". That label is the variable's name.
- Whatever is inside the bottle right now is the variable's value.
- In Python, you fill the bottle like this: age = 21
- The bottle does not care what's inside — the value can be a number, text, or anything else.

A Variable's Value Can Change



Empty it, refill it

- You can pour out the old value and pour in a new one — this is called reassigning.
- The bottle's label (the variable's name) never changes. Only what's inside does.

```
age = 21
print(age)    # 21

age = 22      # same bottle, new value
print(age)    # 22
```

Rules for Naming a Variable



Simple rules

- Can only use letters, numbers, and underscores (_)
- Cannot start with a number — age1 works, 1age does not
- Cannot be a word Python already uses, like print or if
- Is case-sensitive — age, Age, and AGE are three different variables

Good Names vs. Names to Avoid

Good Names

- student_age
- total_price
- is_valid
- favorite_subject
- Explains itself — no guessing needed

Names to Avoid

- x, y, a, b — meaningless on their own
- data1, thing, temp2 — vague, easy to confuse
- A good name makes your code read like a sentence



PART 4

Different Ways to Write a Variable Name

✓ MOST USED IN PYTHON



snake_case

Every word is lowercase, joined by underscores — like a snake sliding between words.

Example: student_age



camelCase

The first word is lowercase, and every word after it starts with a capital letter — like the humps on a camel's back.

Example: studentAge · Common in JavaScript & Java



PascalCase

Every single word starts with a capital letter, including the first one — as if each word is wearing a crown.

Example: StudentAge · Used for class names in Python



CONSTANT_CASE

All capital letters, joined by underscores. Used for values that are locked — they never change while the program runs.

Example: MAX_SCORE · Used for constants in Python

× NOT VALID IN PYTHON



kebab-case

Every word is lowercase, joined by hyphens — like pieces of meat on a kebab skewer.

Example: student-age · Python reads the hyphen as subtraction and breaks

Which Style Does Python Actually Use?



PEP 8: Python's style guide

- Python's official style guide is called PEP 8
- It says: use snake_case for variable and function names
- Why? Lowercase letters with underscores are the easiest style for most people to read quickly
- PascalCase is still used, but only for class names. CONSTANT_CASE is still used, but only for values that never change.

All 5 Styles at a Glance



snake_case

student_age — Python variables (most used)



camelCase

studentAge — JavaScript, Java



PascalCase

StudentAge — Python class names



CONSTANT_CASE

MAX_SCORE — Python constants



kebab-case

student-age — not valid in Python



PART 5

The Four Core Data Types



int — Whole Numbers

Short for "integer." Any whole number, positive or negative, with no decimal point.

Example: age = 21



float — Decimal Numbers

Short for "floating-point number." Any number that has a decimal point.

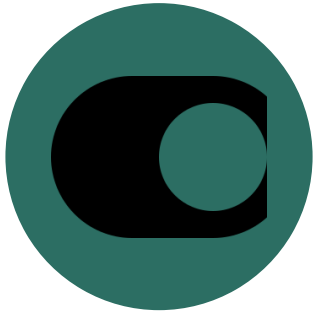
Example: gpa = 3.6



str — Text

Short for "string" — any text, wrapped in quotes. Can be a single letter or a whole sentence.

Example: name = "Ameer"



bool — True or False

Short for "boolean." Only two possible values exist: True or False — like a light switch that's either on or off.

Example: is_enrolled = True

The 4 Core Data Types at a Glance



int

Whole numbers — 21



float

Decimal numbers — 3.6



str

Text — "Ameer"



bool

True or False

Checking a Type: type()

- The type() function asks Python: "what kind of value is this?"
- It always tells you the truth — very useful when your code isn't behaving as expected
- You will use this constantly, especially while learning

```
age = 21
gpa = 3.6
name = 'Ameer'
is_enrolled = True

print(type(age))        # <class 'int'>
print(type(gpa))        # <class 'float'>
print(type(name))       # <class 'str'>
print(type(is_enrolled))  # <class 'bool'>
```

Type Casting — Changing a Value's Type

- Casting means converting a value from one type to another, on purpose
- The three you'll use constantly: int(), float(), str()
- This matters most right after input() — see the next slide

```
age_text = '21'          # this is a str
age_number = int(age_text)   # now it's an int

print(type(age_text))       # <class 'str'>
print(type(age_number))     # <class 'int'>
```

The Three Casting Functions

int()

Converts a value into a whole number

float()

Converts a value into a decimal number

str()

Converts a value into text

input() Always Returns a String

- Whatever the user types, Python treats it as a str — even if it looks like a number
- This surprises almost everyone the first time they see it
- The fix: wrap input() in int() or float() immediately, before using it as a number

```
birth_year = input("Birth year? ")
print(type(birth_year))  # <class 'str'>, even for "2005"

# the fix:
birth_year = int(input("Birth year? "))
print(type(birth_year))  # <class 'int'> now
```

Today's Tasks — Part A: Your Environment

Small tasks — do them in order

1. Create a new virtual environment (conda create --name myenv python=3.11)
2. List all your environments in the terminal (conda env list)
3. Activate your new environment (conda activate myenv)
4. Connect that environment in VS Code (Ctrl+Shift+P → "Python: Select Interpreter")

Today's Tasks — Part B: Python Practice

Small tasks — none of these should take more than a few minutes

- **5** Write and run a "Hello, World!" program using print()
- **6** Print your name on one line, and your age on the next — two separate print() statements
- **7** Store your name, age, and favorite subject in three variables, then print all three together
- **8** Print the type() of each of those three variables
- **9** Take a number using input(), cast it to an int, then print it back

9 small tasks in total — due before our next class



RECAP

A variable is a labeled container. Its value can change — but Python always knows exactly what type that value is.

Next session: conditionals and loops.